第5章 第2节 RAG核心原理（一）-工作流程与语义搜索

- 来源: https://class.imooc.com/lesson/5271#mid=104914
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: RAG核心原理
- 课节: 图文 1-2 RAG核心原理（一）-工作流程与语义搜索

1-2 RAG核心原理（一）-工作流程与语义搜索

第5章 第2节 RAG核心原理（一）-工作流程与语义搜索

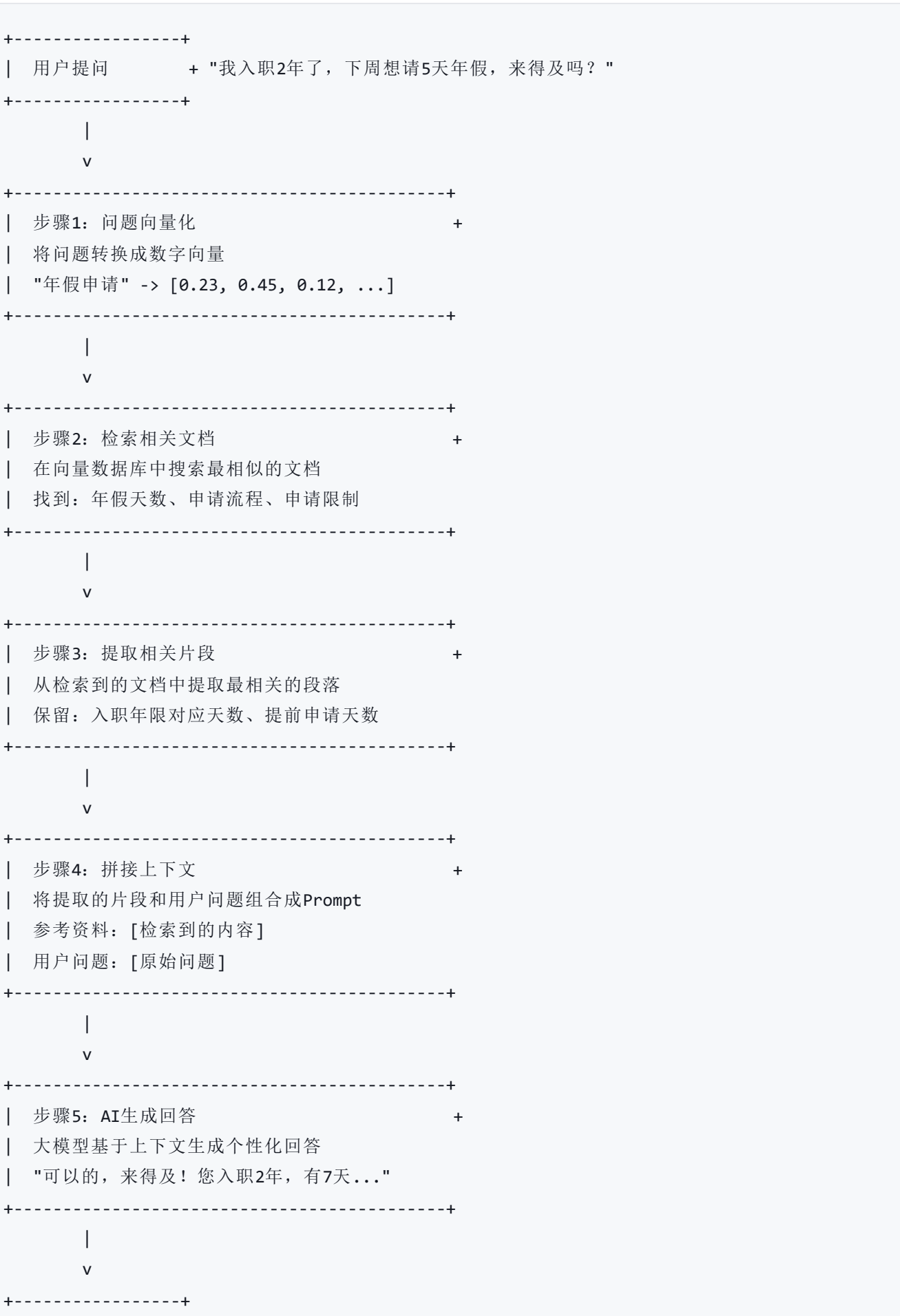
> [!TIP] 阅读指南 在上一节中，我们看到了RAG如何让"死"的知识库变成"活"的智能助手。但你可能会好奇：这背后到底是怎么实现的？AI是如何从成千上万份文档中，精准找到相关内容，并生成自然流畅的回答的？ 这一节，我们就来揭开RAG的技术面纱。

2.1 RAG的工作流程

先从宏观角度，看看RAG完整的工作流程。

假设你的公司有一个智能FAQ系统，员工问："我入职2年了，下周想请5天年假，来得及吗？"系统需要经历以下5个步骤：

完整流程概览


用户提问 + "我入职2年了，下周想请5天年假，来得及吗？"
步骤1：问题向量化
将问题转换成数字向量
"年假申请" -> [0.23, 0.45, 0.12, ...]
步骤2：检索相关文档
在向量数据库中搜索最相似的文档
找到：年假天数、申请流程、申请限制
步骤3：提取相关片段
从检索到的文档中提取最相关的段落
保留：入职年限对应天数、提前申请天数
步骤4：拼接上下文
将提取的片段和用户问题组合成Prompt
参考资料：[检索到的内容]
用户问题：[原始问题]
步骤5：AI生成回答
大模型基于上下文生成个性化回答
"可以的，来得及！您入职2年，有7天..."

```
|  返回给用户       +
+-----------------+
```

这5个步骤环环相扣，缺一不可。接下来，逐个深入理解每个步骤。

2.2 从文字到数字：Embedding的魔法

传统数据库查询时使用特定的SQL来进行查询的。之所以能查询出来是因为数据库的数据通常是“结构化”的。你看MySQL的数据是不是一张张结构规整的表？所以对于SQL查询能从数据库里查出数据来，我们一点都不吃惊。

但RAG不一样，RAG要查询的不是结构化的数据库，而是他也不知道是什么的一堆堆文字，比如一部小说、一堆网页资料等。那他是怎么查询出相关内容的？这非常值得我们探讨一番。

RAG使用的是一种“向量化”的方案，通俗的说就是把文字变成数字。

把文字变成数字

为什么需要向量化

假设，要在一家图书馆找一本关于"人工智能"的书。传统方法是什么？

- 方法1：逐本翻看书名，找包含"人工智能"这几个字的书
- 方法2：按分类号（如TP18）查找，找到"人工智能"类别的所有书

但这两种方法都有问题：

- 方法1太慢，而且找不到用"AI""机器学习"等相关词语的书
- 方法2确实能找到预先分好类的书，但无法判断某本书和要找的书内容上是否真的相关。也许某本书确实在"人工智能"分类下，但是它的内容和你期望的相差 很远。

更聪明的方法是：给每本书一个"特征码"，这个码能代表书的主题、内容、风格。比如：

```
《人工智能基础》      → [0.9, 0.8, 0.1, 0.2, ...]
《机器学习实战》      → [0.85, 0.75, 0.15, 0.25, ...]
《唐诗三百首》        → [0.1, 0.05, 0.9, 0.85, ...]
```

这样，当你问"有没有关于AI的书"时：

- 先把"AI"也转成特征码：[0.88, 0.82, 0.08, 0.18, ...]
- 计算它和每本书特征码的相似度
- 相似度高的就是你要找的书

这就是**Embedding（嵌入）**的核心思想：把文字转换成向量（一组数字），让相似的概念在数字空间里也彼此接近。

Embedding的直观理解

假设我们用2维向量来表示一些词（实际上，不同的Embedding模型维度不同，但为了便于理解，我们用2维来演示）：

```
维度1：动物性（0-1）
                ^
          1.0   |    "猫"(0.9, 0.8)
                |      o  "狗"(0.85, 0.75)
                |        o
          0.5   |                      "苹果"(0.1, 0.2)
                |                         o
                |                       o "橙子"(0.05, 0.15)
          0.0   +--------------------------------> 维度2：食物性（0-1）
                0.0         0.5          1.0
```

可以发现：

- "猫"和"狗"在向量空间里很接近（都是动物）
- "苹果"和"橙子"也很接近（都是食物）
- 但"猫"和"苹果"距离很远（一个是动物，一个是食物）

这就是向量化的魔法：语义相近的词，在向量空间里也相互靠近。

真实的Embedding长什么样

还记得流程图中的步骤1：问题向量化吗？ 看起来好像很难。

但在实际的RAG系统中，这一步通常是通过调用Embedding API来完成的。大多数模型都提供有将文字进行Embedding的API，比如Qwen就有。无论是用户的问题，还是知识库中的文档，都可以经过这个API转换成向量。

实际调用Embedding API会得到：

```
# 文本："年假怎么申请"
embedding_vector = [
    0.0234, -0.0123, 0.0456, 0.0789, -0.0321, 0.0654,
    -0.0198, 0.0432, ... (共1536个数字。不同的模型长度不一样)
]
```

这1536个数字，编码了"年假""申请"这些词的语义信息。

虽然我们人眼看不懂这些数字的含义，但计算机可以通过比较两个向量，判断它们的相似程度——语义相近的文本，向量在数学空间里也会彼此靠近。

2.3 相似度计算：找到最相关的内容

有了向量，下一个问题是：怎么判断两个向量"相似"？

余弦相似度

最常用的方法是余弦相似度（Cosine Similarity）。

不用担心数学公式，我们用一个直观的例子来理解：

想象两人站在原点，各自朝不同方向走：



- 如果俩人走的方向几乎一致（角度接近0度），余弦相似度接近1（非常相似）
- 如果俩人走的方向垂直（角度90度），余弦相似度为0（不相关）
- 如果俩人走的方向相反（角度180度），余弦相似度接近-1（完全相反）

假设用户提问：“我怎么申请年假？”

> [!TIP] 重要说明 Embedding模型不是只能编码单个词，而是可以对整个句子、段落甚至文档进行编码。"我怎么申请年假？"这7个字会被作为一个整体，编码成一个向量。

```
# 步骤1：将用户的完整问题转换为向量
question = "我怎么申请年假？"  # 完整的句子
question_vector = embedding_model.encode(question)
# 结果：[0.23, 0.45, 0.12, ..., 0.33]  (1536维)

# 步骤2：知识库里的文档也都提前转换好了向量
doc1 = "年假申请流程：登录OA系统 → 选择请假申请 → 填写年假类型..."
doc1_vector = [0.25, 0.48, 0.10, ..., 0.35]  # 与问题很接近

doc2 = "公司Wi-Fi密码：办公区密码是Office2024，访客区..."
doc2_vector = [0.01, 0.02, 0.85, ..., 0.90]  # 与问题相差很远

# 步骤3：计算相似度
cosine_similarity(question_vector, doc1_vector)  # → 0.95（非常相似！）
cosine_similarity(question_vector, doc2_vector)  # → 0.12（不相关）
```

为什么文档1相似度高

- 用户问"申请年假"，文档1讲的就是"年假申请流程"
- 两者的向量在1536维空间里的方向几乎一致
- 余弦相似度0.95，说明它们语义高度相关

为什么文档2相似度低

- 用户问"年假"，文档2讲的是"Wi-Fi密码"
- 两者的向量在1536维空间里方向差异很大
- 余弦相似度0.12，说明它们几乎不相关

系统会优先选择相似度高的文档（如文档1），作为回答问题的参考资料。

2.4 检索阶段：在海量文档中找到答案

现在我们知道了如何计算相似度。但如果公司有10万份文档，总不能把每一份都和用户问题比较一遍吧？那太慢了！

所以，单纯的只是把文字转成向量还不行，还需要向量数据库和索引技术，来解决大量向量的管理和查询问题。

向量数据库的作用

普通数据库擅长精确匹配：

```
SELECT * FROM documents WHERE title = '年假申请流程'

1
```

但RAG需要的是相似度搜索：

相似度搜索

```
找出和"年假怎么申请"最相似的5份文档

1
```

向量数据库就是为这个需求设计的：

- 离线阶段： 将所有文档转换成向量 建立特殊的索引结构（如HNSW图） 存储向量和原始文档的对应关系
- 查询阶段： 接收用户问题的向量 利用索引快速定位最相似的候选文档 返回Top-K个最相关的结果

速度对比

- 暴力搜索10万份文档：需要10万次计算，耗时约10秒
- 使用向量数据库：只需几百次计算，耗时约0.01秒

这差不多快了1000倍。

RAG系统通常会取Top-3或Top-5个最相关的文档。

为什么不是Top-1或Top-10?

- Top-1太少：可能遗漏重要信息。比如用户问年假，可能需要"申请流程"+ “天数规定” +"注意事项"三份文档。
- Top-10太多：会引入噪音，增加Token消耗，干扰AI的判断。

Top-10太多

经验规则

- 简单问题：Top-3
- 复杂问题：Top-5

- 专业领域：Top-7

具体数值需要根据实际效果调优。

2.5 ■ 学点英语

中文	English	音标	说明
语义搜索	Semantic Search	/sɪˈmæntɪk sɜːrtʃ/	基于向量空间距离衡量文本语义相似度的搜索方式
余弦相似度	Cosine Similarity	/ˈkoʊsaɪn ˌsɪmə ˈlærəti/	衡量两个向量方向夹角的相似度指标，范围[-1,1]
向量化	Vectorization	/ˌvektərɪˈzeɪʃn/	将文本转换成数字向量表示的过程
欧几里得距离	Euclidean Distance	/juːˈklɪdiən ˈdɪstəns/	多维空间中两点之间的直线距离
Top-K 检索	Top-K Retrieval	/tɑːp keɪ rɪˈtriːvl/	返回相似度最高的前K个检索结果